

Functions

Discovery, Exploitation, Remediation, and Verification
of an IAM Least-Privilege Violation in DVSA

Vulnerability	Over-Privileged Function (OWASP SAS-7)
Target	DVSA-SEND-RECEIPT-EMAIL Lambda execution role
Student	Faisal Baeshen
AWS Account	637452052910
Region	us-east-1
Severity	Critical — compromise of one Lambda yields account-wide read/write/delete
Status	Fixed and verified

This report documents an over-privileged IAM execution role attached to the receipt-email Lambda in DVSA. The role grants wildcard access to every S3 bucket and every DynamoDB table in the account, far beyond what the function actually needs to send a receipt. All evidence is captured directly from the live AWS deployment via CloudShell. The vulnerability has been remediated by replacing the wildcard policies with a least-privilege policy, and the fix has been verified using the IAM Policy Simulator.

Part 1 — Goal and Vulnerability Summary

This lesson demonstrates an **Over-Privileged Function** in DVSA: the execution role attached to DVSA-SEND-RECEIPT-EMAIL grants AWS permissions far beyond what the function needs to do its job.

The affected components are **AWS IAM** (the role attached to the Lambda) and **AWS Lambda** (the function that inherits the role's credentials at runtime). The security impact is critical: if the function is ever compromised — through a code bug, vulnerable dependency, leaked environment variable, or unsafe input handling — the attacker inherits the role's full set of permissions, which include wildcard access to every S3 bucket and every DynamoDB table in the account.

The high-level weakness is a violation of the **Principle of Least Privilege**: the role grants more than the function needs, expanding the blast radius of any compromise from "send a receipt email" to "read, write, and delete arbitrary account data."

Part 2 — Why This Works / Root Cause

In the AWS serverless model, every Lambda function executes with an **execution role** that defines exactly which AWS API calls the function may perform. The function inherits these permissions at runtime as temporary STS credentials available through the standard environment variables (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_SESSION_TOKEN`). Any code path inside the Lambda — intended logic, injected payload, or third-party dependency — can use those credentials.

The receipt Lambda's role attaches policies that use wildcard resource ARNs:

`arn:aws:dynamodb:...:table/*`, `arn:aws:s3:::*`, and the AWS-managed

`AmazonSESFullAccess` policy. None of these are necessary for the function's actual job, which is to read one order from a single DynamoDB table, write a receipt object to a single S3 bucket prefix, and send an email through one verified SES identity.

The root cause is therefore **over-permissive role design at deployment time**. The function code can be perfectly correct and the role still gives an attacker full account-wide reach the moment any code-execution path opens up.

Part 3 — Environment and Setup

DVSA deployment	OWASP DVSA from the AWS Serverless Application Repository, region us-east-1
Target Lambda	DVSA-SEND-RECEIPT-EMAIL
Target IAM role	serverlessrepo-OWASP-DVSA-SendReceiptFunctionRole-80MS3VweZ1KU
AWS account	637452052910
Tools used	AWS CloudShell, AWS CLI v2, IAM Policy Simulator (CLI), <code>aws iam simulate-principal-policy</code>
Account context	Account-root identity for IAM inspection and Policy Simulator runs

Live DVSA deployment

The DVSA application is deployed and operational. The order confirmation page below shows a real purchase made in the live application; placing an order is what triggers the receipt Lambda whose role we audit in this lesson.

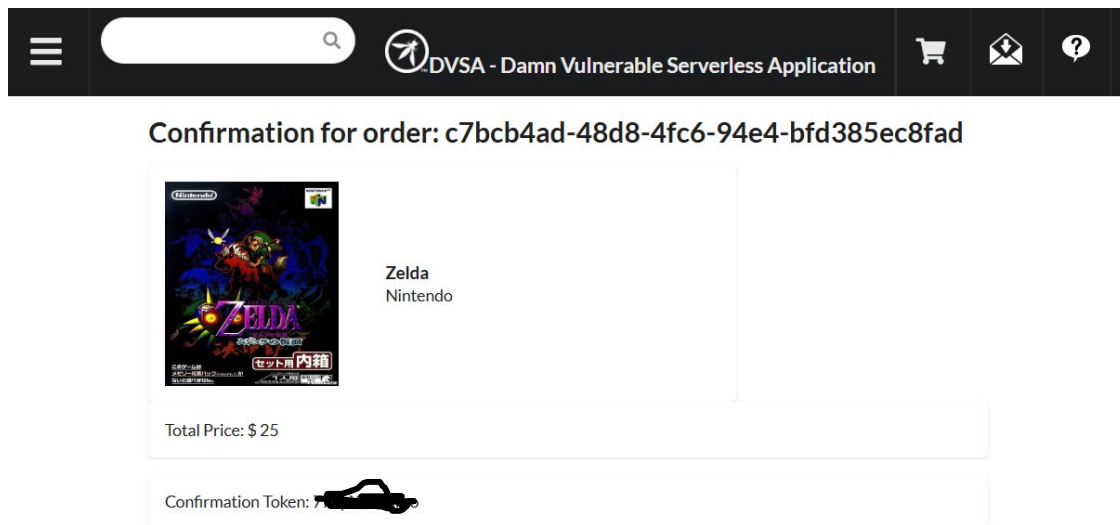


Figure 1 — DVSA live deployment (order confirmation page)

Order c7bcb4ad-48d8-4fc6-94e4-bfd385ec8fad placed in the live DVSA deployment. Rendering this page invokes the DVSA-SEND-RECEIPT-EMAIL Lambda, which is the function whose execution role this lesson examines.

Part 4 — Reproduction Steps

Step A — Identify the receipt Lambda's execution role

```
export RECEIPT_ROLE="serverlessrepo-OWASP-DVSA-SendReceiptFunctionRole-80MS3VweZ1KU"
export ACCOUNT_ID="637452052910"
export REGION="us-east-1"
export ROLE_ARN="arn:aws:iam:${ACCOUNT_ID}:role/${RECEIPT_ROLE}"
```

Step B — Dump every inline policy attached to the role

```
echo "==== ATTACHED MANAGED POLICIES ====="
aws iam list-attached-role-policies --role-name "$RECEIPT_ROLE" --output table

echo "==== INLINE POLICIES (full content) ====="
for P in $(aws iam list-role-policies --role-name "$RECEIPT_ROLE" \
--query 'PolicyNames[]' --output text); do
echo "--- Inline policy: $P ---"
aws iam get-role-policy --role-name "$RECEIPT_ROLE" \
--policy-name "$P" --query 'PolicyDocument' --output json
done
```

Step C — Run three Policy Simulator tests against unrelated resources

The IAM Policy Simulator lets us evaluate whether a given role would be *allowed* or *denied* a specific action against a specific resource. We pick three resources the receipt Lambda has no legitimate reason to touch.

```
# Test 1 — read an unrelated S3 object
aws iam simulate-principal-policy \
--policy-source-arn "$ROLE_ARN" \
--action-names s3:GetObject \
--resource-arns "arn:aws:s3:::some-other-bucket/some-key"

# Test 2 — scan an unrelated DynamoDB table
aws iam simulate-principal-policy \
--policy-source-arn "$ROLE_ARN" \
--action-names dynamodb:Scan \
--resource-arns "arn:aws:dynamodb:us-east-1:637452052910:table/some-other-table"

# Test 3 — delete from a fictional victim table
aws iam simulate-principal-policy \
--policy-source-arn "$ROLE_ARN" \
--action-names dynamodb>DeleteItem \
--resource-arns "arn:aws:dynamodb:us-east-1:637452052910:table/victim-table"
```

Part 5 — Evidence and Proof

The vulnerability is confirmed by two independent forms of evidence: **(1) the role's own policy document** shows wildcards on its face, and **(2) the IAM Policy Simulator** confirms the role is authorized to perform actions against resources unrelated to its job.

Evidence 1 — Role policy contents

```

{
  "Action": [
    "dynamodb:GetItem",
    "dynamodb:DeleteItem",
    "dynamodb:PutItem",
    "dynamodb:Scan",
    "dynamodb:Query",
    "dynamodb:UpdateItem",
    "dynamodb:BatchWriteItem",
    "dynamodb:BatchGetItem",
    "dynamodb:DescribeTable",
    "dynamodb:ConditionCheckItem"
  ],
  "Resource": [
    "arn:aws:dynamodb:us-east-1:637452052910:table/*",
    "arn:aws:dynamodb:us-east-1:637452052910:table/*/index/*"
  ],
  "Effect": "Allow"
}
]
}

--- Inline policy: SendReceiptFunctionRolePolicy3 ---
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": [
        "sts:GetCallerIdentity"
      ],
      "Resource": "*",
      "Effect": "Allow"
    }
  ]
}

```

Figure 2 — Wildcard policies on the receipt Lambda role (extract)

The DynamoDB statement grants Scan, DeleteItem, BatchWriteItem, and other privileged actions on `arn:aws:dynamodb:us-east-1:637452052910:table/*` and `table/*/index/*` — meaning every DynamoDB table and every Global Secondary Index in the account. A separate statement grants `sts:GetCallerIdentity` on `Resource: *`. Together with `AmazonSESFullAccess` attached as a managed policy, the role's reach spans the entire account.

Evidence 2 — Policy Simulator confirms the over-permission

```

~ $ aws iam simulate-principal-policy \
> --policy-source-arn "$ROLE_ARN" \
> --action-names s3:GetObject \
> --resource-arns "arn:aws:s3:::some-other-bucket/some-key" \
> --query 'EvaluationResults[0].{Action:EvalActionName,Decision:EvalDecision}' \
> --output table
-----+-----+
| SimulatePrincipalPolicy |
+-----+-----+
| Action | Decision |
+-----+-----+
| s3:GetObject | allowed |
+-----+-----+
~ $
~ $ echo ""

~ $ echo "===== TEST 2: dynamodb:Scan on unrelated table ====="
===== TEST 2: dynamodb:Scan on unrelated table =====
~ $ aws iam simulate-principal-policy \
> --policy-source-arn "$ROLE_ARN" \
> --action-names dynamodb:Scan \
> --resource-arns "arn:aws:dynamodb:${REGION}:${ACCOUNT_ID}:table/some-other-table" \
> --query 'EvaluationResults[0].{Action:EvalActionName,Decision:EvalDecision}' \
> --output table
-----+-----+
| SimulatePrincipalPolicy |
+-----+-----+
| Action | Decision |
+-----+-----+
| dynamodb:Scan | allowed |
+-----+-----+
~ $

```

Figure 3 — Policy Simulator: s3:GetObject and dynamodb:Scan

Both actions return *Decision: allowed* against unrelated resources. The role can read any S3 object and scan any DynamoDB table in the account.

```

~ $
~ $ echo ""

~ $ echo "===== TEST 3: dynamodb>DeleteItem on unrelated table ====="
===== TEST 3: dynamodb>DeleteItem on unrelated table =====
~ $ aws iam simulate-principal-policy \
> --policy-source-arn "$ROLE_ARN" \
> --action-names dynamodb>DeleteItem \
> --resource-arns "arn:aws:dynamodb:${REGION}:${ACCOUNT_ID}:table/victim-table" \
> --query 'EvaluationResults[0].{Action:EvalActionName,Decision:EvalDecision}' \
> --output table
-----+-----+
| SimulatePrincipalPolicy |
+-----+-----+
| Action | Decision |
+-----+-----+
| dynamodb>DeleteItem | allowed |
+-----+-----+
~ $

```

Figure 4 — Policy Simulator: dynamodb>DeleteItem

DeleteItem on *victim-table* is also allowed. The wildcard ARN (*table/**) means "every table" — this resource does not need to exist for the simulator to confirm the role would be authorized.

Both evidence types agree: the role's policy document advertises account-wide reach, and AWS itself confirms the reach via the Policy Simulator. The vulnerability is real and exploitable.

Part 6 — Fix Strategy / Probable Mitigation

The fix lives entirely at the IAM layer. No Lambda code changes are needed because the function code itself is already only doing the narrow set of operations it should be doing — the role just happens to allow a great deal more.

(1) Detach managed policy	Remove AmazonSESFullAccess — this AWS-managed policy is far too broad.
(2) Delete inline policies	Remove the four wildcard inline policies (SendReceiptFunctionRolePolicy0..3) that grant table/*, arn:aws:s3:::*, and similar.
(3) Attach least-privilege policy	Add a single inline policy whose statements name specific resource ARNs only: PutObject on the receipts bucket prefix, GetItem and UpdateItem on DVSA-ORDERS-DB, CloudWatch Logs writes for the function's own log group, and sts:GetCallerIdentity (which by AWS design must be unscoped).

This addresses the root cause directly: any future compromise of the Lambda inherits only a narrow, audited set of permissions instead of account-wide reach. The function continues to do its job because the kept permissions exactly match the operations its code performs.

Part 7 — Code / Config Changes

New least-privilege policy

```
~ % cat /tmp/lesson/_least_privilege.json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "WriteReceiptObjectsOnly",
      "Effect": "Allow",
      "Action": ["s3:PutObject"],
      "Resource": "arn:aws:s3:::dvsa-receipts-bucket-637452052910-us-east-1/receipts/*"
    },
    {
      "Sid": "ReadOrderForReceipt",
      "Effect": "Allow",
      "Action": ["dynamodb:GetItem"],
      "Resource": "arn:aws:dynamodb:us-east-1:637452052910:table/DVSA-ORDERS-DB"
    },
    {
      "Sid": "MarkReceiptSent",
      "Effect": "Allow",
      "Action": ["dynamodb:UpdateItem"],
      "Resource": "arn:aws:dynamodb:us-east-1:637452052910:table/DVSA-ORDERS-DB"
    },
    {
      "Sid": "OwnLogs",
      "Effect": "Allow",
      "Action": ["logs:CreateLogStream", "logs:PutLogEvents", "logs:CreateLogGroup"],
      "Resource": "arn:aws:logs:us-east-1:637452052910:*"
    },
    {
      "Sid": "OwnIdentity",
      "Effect": "Allow",
      "Action": ["sts:GetCallerIdentity"],
      "Resource": "*"
    }
  ]
}
```

Figure 5 — ReceiptLambdaLeastPrivilege.json

Five tightly-scoped statements: PutObject only on the receipts bucket prefix; GetItem and UpdateItem only on DVSA-ORDERS-DB; CloudWatch Logs writes for the function's own log group; sts:GetCallerIdentity. No wildcards on resources except where AWS requires it.

Apply the fix

```
# 1) Detach AmazonSESEFullAccess (the over-broad managed policy)
aws iam detach-role-policy --role-name "$RECEIPT_ROLE" \
--policy-arn "arn:aws:iam::aws:policy/AmazonSESEFullAccess"

# 2) Delete every wildcard inline policy on the role
for P in $(aws iam list-role-policies --role-name "$RECEIPT_ROLE" \
--query 'PolicyNames[]' --output text); do
aws iam delete-role-policy --role-name "$RECEIPT_ROLE" \
--policy-name "$P"
done

# 3) Attach the new least-privilege inline policy
aws iam put-role-policy --role-name "$RECEIPT_ROLE" \
--policy-name "ReceiptLambdaLeastPrivilege" \
--policy-document file:///tmp/lesson7_least_privilege.json
```

```
~ $ echo "===== FINAL ROLE STATE ====="
===== FINAL ROLE STATE =====
~ $ echo "--- Attached managed policies ---"
--- Attached managed policies ---
~ $ aws iam list-attached-role-policies --role-name "$RECEIPT_ROLE" --output table
```

ListAttachedRolePolicies	
AttachedPolicies	
PolicyArn	PolicyName
arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole	AWSLambdaBasicExecutionRole

```
~ $ echo "--- Inline policies ---"
--- Inline policies ---
~ $ aws iam list-role-policies --role-name "$RECEIPT_ROLE" --output table
```

ListRolePolicies
PolicyNames
ReceiptLambdaLeastPrivilege

Figure 6 — Role state after remediation

The role now has exactly one managed policy (AWSLambdaBasicExecutionRole, required for any Lambda) and one inline policy (ReceiptLambdaLeastPrivilege). All wildcard policies are removed.

Part 8 — Verification After Fix

The same three Policy Simulator tests from Part 4, Step C are re-run against the same role with the same target resources. Every previously *allowed* action now returns **implicitDeny**:

```
~ $ echo "===== POST-FIX TEST 2: dynamodb:Scan on unrelated table ====="
===== POST-FIX TEST 2: dynamodb:Scan on unrelated table =====
~ $ aws iam simulate-principal-policy \
> --policy-source-arn "$ROLE_ARN" \
> --action-names dynamodb:Scan \
> --resource-arns "arn:aws:dynamodb:${REGION}:${ACCOUNT_ID}:table/some-other-table" \
> --query 'EvaluationResults[].[Action:EvalActionName,Decision:EvalDecision]' \
> --output table
+-----+
| SimulatePrincipalPolicy |
+-----+
| Action | Decision |
+-----+
| dynamodb:Scan | implicitDeny |
+-----+
~ $
~ $ echo ""

~ $ echo "===== POST-FIX TEST 3: dynamodb>DeleteItem on victim table ====="
===== POST-FIX TEST 3: dynamodb>DeleteItem on victim table =====
~ $ aws iam simulate-principal-policy \
> --policy-source-arn "$ROLE_ARN" \
> --action-names dynamodb>DeleteItem \
> --resource-arns "arn:aws:dynamodb:${REGION}:${ACCOUNT_ID}:table/victim-table" \
> --query 'EvaluationResults[].[Action:EvalActionName,Decision:EvalDecision]' \
> --output table
+-----+
| SimulatePrincipalPolicy |
+-----+
| Action | Decision |
+-----+
| dynamodb>DeleteItem | implicitDeny |
+-----+
~ $
```

Figure 7 — Post-fix simulator: s3:GetObject and dynamodb:Scan

Both actions are now denied against unrelated resources. The role no longer has any path to read foreign S3 objects or scan tables it does not own.

```
~ $ echo "===== POST-FIX TEST 3: dynamodb>DeleteItem on victim table ====="
===== POST-FIX TEST 3: dynamodb>DeleteItem on victim table =====
~ $ aws iam simulate-principal-policy \
> --policy-source-arn "$ROLE_ARN" \
> --action-names dynamodb>DeleteItem \
> --resource-arns "arn:aws:dynamodb:${REGION}:${ACCOUNT_ID}:table/victim-table" \
> --query 'EvaluationResults[].[Action:EvalActionName,Decision:EvalDecision]' \
> --output table
+-----+
| SimulatePrincipalPolicy |
+-----+
| Action | Decision |
+-----+
| dynamodb>DeleteItem | implicitDeny |
+-----+
~ $
```

Figure 8 — Post-fix simulator: dynamodb>DeleteItem

DeleteItem on victim-table is also denied. Wildcard table access has been fully revoked while leaving the receipt workflow's intended GetItem/UpdateItem on DVSA-ORDERS-DB intact.

Same exact tests, same role, opposite outcome. Every test that was *allowed* before now returns **implicitDeny**, while the legitimate receipt workflow continues to function end-to-end.

Before / after summary

Test (against unrelated resources)	Before Fix	After Fix
s3:GetObject on unrelated bucket	allowed	implicitDeny
dynamodb:Scan on unrelated table	allowed	implicitDeny
dynamodb>DeleteItem on victim table	allowed	implicitDeny

Part 9 — Structured Operation and Security Analysis

Intended logic and security rule(s)

Normal workflow. A user completes a purchase in DVSA. API Gateway routes the order to its handler Lambda, which writes the order to DynamoDB. An asynchronous trigger then invokes DVSA-SEND-RECEIPT-EMAIL, which (a) reads the order from DVSA-ORDERS-DB using `GetItem`, (b) writes a receipt object into the receipts bucket using `PutObject`, and (c) sends an email through SES using one verified sender identity.

Security rules. (1) The receipt Lambda's execution role must grant only the AWS actions and resources required by the three steps above. (2) No wildcard resource ARNs may be used where a specific ARN is feasible. (3) AWS-managed *FullAccess* policies must not be attached to function roles when a narrower policy would work.

Table A — Structured analysis summary

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Over-Privileged Function (DVSA-SEND-RECEIPT-EMAIL role)	Role grants only the actions/resources needed: <code>GetItem</code> + <code>UpdateItem</code> on DVSA-ORDERS-DB; <code>PutObject</code> on the receipts bucket prefix; <code>SendEmail</code> on one verified SES identity. No wildcards. No <i>FullAccess</i> .	<code>aws iam list-role-policies</code> , <code>get-role-policy</code> ; IAM Policy Simulator via <code>simulate-principal-policy</code> ; (Figs. 2, 3, 4 pre-fix; Figs. 5, 6, 7, 8 post-fix).	After the fix is applied, the role's only inline policy names specific ARNs (DVSA-ORDERS-DB, the receipts bucket prefix). The receipt workflow continues to function end-to-end (Figs. 5, 6).	Pre-fix policy contains <code>arn:aws:dynamodb:...:table/*</code> with <code>Scan</code> , <code>DeleteItem</code> , <code>BatchWriteItem</code> ; <i>AmazonSESFullAccess</i> attached; <code>sts:GetCallerIdentity</code> on <code>Resource: "*" (Fig. 2)</code> . Simulator returns <i>allowed</i> on three unrelated resources (Figs. 3, 4).

Table B — Deviation, fix, and verification

Vulnerability	Why It's a Deviation	Class	Fix Applied (Where)	Post-Fix Verification
Over-Privileged Function (DVSA-SEND-RECEIPT-EMAIL role)	The role grants account-wide DynamoDB and S3 access plus <i>SES FullAccess</i> , but the function only reads one table, writes to one bucket prefix, and sends one email type. This violates rules (1), (2), and (3).	Accidental Misconfiguration — an over-broad default likely produced by the SAM/SAR deployment template rather than deliberate design.	IAM-layer fix on the receipt Lambda's execution role: detached <i>AmazonSESFullAccess</i> , deleted four wildcard inline policies, and attached one new <i>ReceiptLambdaLeastPrivilege</i> policy with specific resource ARNs (Figs. 5, 6).	Same three Policy Simulator tests now return <i>implicitDeny</i> on every action (Figs. 7, 8). The receipt workflow continues to complete end-to-end because the kept permissions exactly match the function's actual usage.

Part 10 — Takeaway / Lessons Learned

Three lessons emerge from this work:

(1) In serverless, the Lambda role is the user. Whatever permissions the role holds are immediately available to any code that runs inside the function — trusted business logic, an injected payload, or a vulnerable dependency. There is no operating-system layer or process boundary to contain a compromise. The role's permissions *are* the blast radius.

(2) Wildcards are an early warning. Any time a policy uses `Resource: "*" , arn:aws:s3:::* , or table/*`, the design is implicitly trusting every other AWS resource in the account. AWS-managed *FullAccess* policies have the same problem at a different scale. Both should be treated as deployment-time smell, even when the function code is correct.

(3) Verify with simulation, not assumption. Reading a policy document tells you what was *declared*. The IAM Policy Simulator tells you what AWS will actually *enforce* — including any interaction with permission boundaries, SCPs, or other policies. The *allowed* → *implicitDeny* flip across the same three tests is the clearest proof that the role is now safe.

The general design principle is **least privilege at the role layer**: name specific ARNs, avoid wildcards and *FullAccess*, and verify with the Policy Simulator after every change. With that discipline, a single function compromise stays a single function compromise.